Tema 3.
Introducción a Javascript.

Historia

- El lenguaje fue inventado por Brendan Eich en la empresa Netscape Communications.
- Apareció por primera vez en Netscape Navigator 2.0
- Los autores inicialmente lo llamaron Mocha y más tarde LiveScript.
- El cambio de nombre de LiveScript a JavaScript coincidió con el anuncio de Netscape que añadía soporte a la tecnología Java en Netscape Navigator.
- Microsoft llamó JScript a su implementación de JavaScript en Internet Explorer.

¿Qué es JavaScript?

- JavaScript está diseñado para añadir interactividad a las páginas HTML.
- JavaScript es un lenguaje de scripts (un lenguaje de scripts es un lenguaje de programación ligero).
- Un JavaScript consiste de líneas de código ejecutable.
- Un JavaScript por lo general está incluido directamente en las páginas HTML.
- JavaScript es un lenguaje interpretado (los scripts se ejecutan sin previa compilación).
- Todos pueden usar JavaScript sin comprar una licencia.

¿Es lo mismo Java y JavaScript?

- ¡NO!
- Java y JavaScript son dos lenguajes totalmente diferentes en concepto y diseño.
- Java (desarrollado por Sun Microsystems) es un lenguaje de programación poderoso y mucho más complejo a la altura de C y C++.

JavaScript ≠



¿Qué puede hacer JavaScript?

- JavaScript les da a los diseñadores de HTML una herramienta de programación.
- JavaScript puede poner texto dinámico en una página.
- JavaScript puede reaccionar a eventos.
- JavaScript puede leer y escribir elementos HTML.
- **JavaScript se puede utilizar para validar datos.**
- JavaScript se puede usar para detectar el navegador del visitante.
- JavaScript se puede usar para crear cookies.

Sintaxis

- No se toman en cuenta los espacios en blanco y las nuevas líneas.
- Se distinguen las mayúsculas y minúsculas.
- **No se define el tipo de variables.**
- No es necesario terminar cada secuencia con punto y coma (pero es altamente recomendado).
- Se pueden incluir comentarios.

Posibilidades y Limitaciones

- Entorno limitado
- Solo cierra ventanas que el script mismo haya abierto
- Ventanas fuera de la vista del usuario (depende del navegador)
- No se puede acceder a archivos del sistema
- No se puede acceder a preferencias del navegador
- Se puede detener el script si toma demasiado tiempo

Posibilidades y Limitaciones

- Crear y leer cookies dentro del mismo dominio
- Acceso limitado a objetos potencialmente peligrosos
- Confirmación del usuario antes de realizar ciertas acciones
- Elementos descargados de otros servidores bloqueados

Un ejemplo sencillo

- HelloWorld en JavaScript

```
<html>
<body>
<script type="text/javascript">
document.write("Hello World!")
</script>
</body>
</html>
```

¿Cómo manejar navegadores antiguos?

- Los navegadores que no soportan JavaScript despliegan el script como contenido de la página, para prevenir esto, podemos usar la etiqueta de comentarios de HTML.

```
<script type="text/javascript">
//<!–
document.write("Hello World!")
//-->
</script>
```

¿Dónde poner los JavaScripts?

- En el encabezado del archivo HTML (sección <head> </head>):
  - El script se carga antes de que alguien lo use. Recomendable para validaciones o funciones que se llamen desde otras secciones.

- En el cuerpo del archivo HTML (sección <body> </body>):
  - Estos scripts se ejecutan cuando la página se carga y generan el contenido de la misma.

Usando un JavaScript externo

- Alguna vez es necesario ejecutar un mismo script en varias páginas.
- Para simplificar esto puedes escribir un JavaScript en un archivo externo. Guarda el archivo externo con extensión .js. El script externo no debe tener la etiqueta <script>.

```
<html>
<head>
<script src="xxx.js"></script>
</head>
<body>
</body>
</html>
```

Variables

- Reglas para los nombres de variables:
  - Son sensibles a mayúsculas.
  - Deben comenzar con una letra o un subguión (_) o los símbolos $ (dólar).
  - Solo existe un tipo de variable y puede contener cadenas, números enteros y/o de punto flotante.
- Declaración de variables: Dos formas

```
var nombre = “Salomon”
nombre = “Salomon”
```

- Comentarios
  - // Comentario de una línea
  - /* Comentario de múltiples líneas */

Variables

```
var numero1 = 3;
var numero2 = 2;
var resultado = numero1 + numero2;

var num1 = 1;
var num2 = 2;
resultado = num1 + num2
var num1 = 1;
var num2;
num2 = 2;
var resultado = num1 +
```

Tipos de Datos

- Mecanismos de Escape
  - \n Una nueva línea
  - \t Un tabulador
  - \' Una comilla simple
  - \" Una comilla doble
  - \\ Una barra inclinada
- JavaScript define cinco tipos primitivos:
  - undefined, null, boolean, number y string.
    - ✓ Undefined corresponde a las variables que han sido definidas y todavía no se les ha asignado un valor
    - ✓ El tipo null se suele utilizar para representar objetos que en ese momento no existen.
    - ✓ Number permite definir variables de tipo numérico independientemente de si el valor es entero o decimal
      - Var variable1 = new Number(10);
  - El operador typeof para averiguar el tipo de una variable.

Tipos de Datos

- Conversión entre tipos de variables
  - JavaScript es un lenguaje de programación "no tipado", una misma variable puede guardar diferentes tipos de datos a lo largo de la ejecución de la aplicación.
  - Pero en ocasiones es necesario convertir o asegurar que una variable es de un tipo de dato
    - ✓ toString() que permite convertir variables de cualquier tipo a variables de cadena de texto
    - ✓ parseInt() y parseFloat() convierten la variable que se le indica en un número entero o un número decimal respectivamente.

```
var variable1 = "hola";
parseInt(variable1);  // devuelve NaN
var variable2 = "34";
parseInt(variable2);  // devuelve 34
var variable3 = "34hola23";
parseInt(variable3);  // devuelve 34
var variable4 = "34.23";
parseInt(variable4);  // devuelve 34
```

Sentencia if

```
<script type="text/javascript">
//Escribir mensaje de ¡Buenos días!
//si la hora es menor a las 12

var d=new Date()
var hora=d.getHours()
if (hora<12)
{
   document.write("<b>¡Buenos días!</b>")
}

</script>
```

if…else

```
<script type="text/javascript">
//Escribir mensaje de ¡Buenos días!
//si la hora es menor a las 12

var d=new Date()
var hora=d.getHours()
if (hora<12)
{
   document.write("<b>¡Buenos días!</b>")
}
else
{
   document.write("<b>¡Buenas tardes!</b>")
}

</script>
```

if…else if…else

```
<script type="text/javascript">
//Escribir mensaje de ¡Buenos días!
//si la hora es menor a las 12

var d=new Date()
var hora=d.getHours()
if (hora<12)
{
   document.write("<b>¡Buenos días!</b>")
}
else if (hora<19)
{
   document.write("<b>¡Buenas tardes!</b>")
}
else
{
   document.write("<b>¡Buenas noches!</b>")
}


</script>
```

switch

```
<script type="text/javascript">
//Recibes un mensaje diferente dependiendo
//del día que sea. Cabe señalar que el Domingo=0,
//Lunes=1, Martes=2, etc.
var d=new Date()
theDay=d.getDay()
switch (theDay)
{
case 5:
   document.write(“Por fin es Viernes")
   break
case 6:
   document.write(“Super Sábado")
   break
case 0:
   document.write(“Domingo de Flojera")
   break
default:
   document.write(“Espero que pronto llegue el fin de semana")
}
</script>
```

Estructura for

```
for (var=valordeinicio;var<=valorfinal;var=var+incremento)
{
Código a ejecutar
}
```

Ejemplo:

```
<html>
<body>
<script type="text/javascript">
var i=0
for (i=0;i<=10;i++)
{
   document.write(“El número es " + i)
   document.write("<br />")
}
</script>
</body>
</html>
```

Estructura for

```
var mensaje = "Hola, estoy dentro de un ciclo";

for(var i = 0; i < 5; i++) {
  alert(mensaje);
}
```

Estructura while

```
var resultado = 0;
var numero = 100;
var i = 0;

while(i <= numero) {
  resultado += i;
  i++;
}

alert(resultado);
```

Estructura do...while

- El código se ejecutará al menos una vez

```
do {
   ...
} while (condicion);
```

- Break y Continue: Permiten manipular el comportamiento normal de un ciclo.
  - break permite terminar de forma abrupta un ciclo.
  - continue permite saltarse algunas repeticiones del ciclo.

Estructura do...while

```
var resultado = 1;
var numero = 5;

do {
  resultado *= numero;
  numero--;
} while(numero > 0);

alert(resultado);
```

Operadores aritméticos

Operador	Descripción	Ejemplo	Resultado	
+	Suma	x=2		4
		y=2		
		x+y		
-	Resta	x=5		3
		y=2		
		x-y		
*	Multiplicación	x=5		20
		y=4		
		x*y		
/	División	15/5		3
		5/2		2.5
%	Módulo	5%2		1
		10%8		2
		10%2		0
++	Incremento	x=5	x=6	
		x++		
--	Decremento	x=5	x=4	
		x--		

Operadores de asignación

Operador	Ejemplo	Es lo mismo que
=	x=y	x=y
+=	x+=y	x=x+y
-=	x-=y	x=x-y
*=	x*=y	x=x*y
/=	x/=y	x=x/y
%=	x%=y	x=x%y

Operadores de comparación

Operador	Descripción	Ejemplo
==	igual a	5==8 regresa falso
===	igual a (verifica valor y tipo)	x=5 y="5" x==y regresa verdadero x===y regresa falso
!=	diferente de	5!=8 regresa verdadero
>	mayor que	5>8 regresa falso
<	menor que	5<8 regresa verdadero
>=	mayor o igual	5>=8 regresa falso
<=	menor o igual	5<=8 regresa verdadero

Operadores Lógicos

Operador	Descripción	Ejemplo
&&	y	x=6 y=3 (x < 10 && y > 1) regresa verdadero
||	o	x=6 y=3 (x==5 || y==5) regresa falso
!	no	x=6 y=3 !(x==y) regresa verdadero

Mensajes

- Mensaje de alerta
  - alert(“alerta")
- Mensaje de confirmación
  - confirm(“confirmación")
- Mensaje de pregunta
  - prompt(“pregunta",“valor por defecto")

Funciones

- Una función es un bloque de código que se ejecuta por un evento o cuando es llamada:

```
<html>
<head>
<script type="text/javascript">
        function displaymessage()
        { alert("Hello World!") }
</script>
</head>

<body>
  <form>
        <input type="button" value="Click me!"
                onclick="displaymessage()" >
  </form>
</body>
</html>
```

Funciones

```
function hola(msg) {
   alert('Hola ' + msg);
}

hola('Mundo');
```

```
function suma(num1, num2) {
  return num1 + num2;
}

var numero1 = 5;
var numero2 = 6;
var suma = suma(numero1, numero2);
```

- Funciones anónimas

```
document.onmousedown = function()
{ alert("Presionaste el boton"); }
```

Arrays

- Se puede acceder al índice a través de una variable
  - `var miArray = new Array(10);`
  - `var miArray = [];`

```
for(i=0; i<6; i++) {
   document.writeln(x[i]+'<br>);
}
0
1
2
3
4
5
```

Inicializando un Array

- Inicializar un Array con datos predefinidos:

```
var miArray = ['Enero', 'Febrero', 'Marzo'];

document.writeln('0>'+miArray[0]+'<BR>');
       // Salida: 0>Enero

document.writeln('1>'+miArray[1]+'<BR>');
       // Salida: 1>Febrero

document.writeln('2>'+miArray[2]+'<BR>');
       // Salida: 2>Marzo
```

Inicializando un Array

- Inicializarlo con datos después de haber sido creado:

```
var miArray = [];
miArray[0] = 'Enero';
miArray[1] = 'Febrero';
miArray[2] = 'Marzo';
document.writeln(miArray[0]+'<BR>'); // Enero
document.writeln(miArray[1]+'<BR>'); // Febrero
document.writeln(myArray[2]+'<BR>'); // Marzo
```

Arrays Multidimensionales

```
var x=[0,1,2,3,4,5];
var y=[x];
document.writeln(y[0][3]); // Salida: 2
```

Los Arrays son pasados por Referencia

```
var miArray = [ 'cero', 'uno', 'dos', 'tres' ];
document.writeln(miArray[1]); // Salida: uno

function pasadoPorReferencia(refArray) {
   refArray[1] = 'cambio';
}

pasadoPorReferencia(miArray);
document.writeln(miArray[1]); // Salida: cambio
```

Los Arrays son asignados por Referencia

```
var miArray = [ 'cero', 'uno', 'dos' ];
var nuevoArray = miArray;

nuevoArray[1] = 'cambio';

document.writeln(miArray[1]); // Salida: cambio
```

Pasando Arrays por valor

```
var miArray = [ 'cero', 'uno', 'dos' ];
var nuevoArray = miArray.slice();

nuevoArray[1] = 'cambio';

document.writeln(miArray[1]); //salida: uno

function pasadoPorReferencia(refArray) {
   refArray[1] = 'cambio';
}

pasadoPorReferencia(miArray.slice());
document.writeln(miArray[1]); // Salida: uno
```

Funciones. Texto

- lenght calcula la longitud de una cadena.
  - var mensaje = "Hola Mundo";
  - var numLetras = mensaje.length; //numeroLetras = 10
- + concatena cadenas
  - var mensaje1 = "Hola";
  - var mensaje2 = " Mundo";
  - var mensaje = mensaje1 + mensaje2; // mensaje = "Hola Mundo"
- concat() concatena cadenas
  - var mensaje1 = "Hola";
  - var mensaje2 = mensaje1.concat(" Mundo");
  - // mensaje2 = "Hola Mundo"

Funciones. Texto

- toUpperCase() transforma a mayúsculas todos los caracteres
  - var mensaje1 = "Hola";
  - var mensaje2 = mensaje1.toUpperCase(); // mensaje2 = "HOLA"

- toLowercase() transforma a minúsculas todos los caracteres
  - var mensaje1 = "HolA";
  - var mensaje2 = mensaje1.toLowerCase(); // mensaje2 = "hola"

Funciones. Texto

- charAt(posicion)
  - var mensaje = "Hola";
  - var letra = mensaje.charAt(0); // letra = H
  - letra = mensaje.charAt(2); // letra = l
- indexOf(caracter)
  - var mensaje = "Hola";
  - var posicion = mensaje.indexOf('a'); // posicion = 3
  - posicion = mensaje.indexOf('b'); // posicion = -1
- lastIndexOf(caracter)
  - var mensaje = "Hola";
  - var posicion = mensaje.lastIndexOf('a'); // posicion = 3
  - posicion = mensaje.lastIndexOf('b'); // posicion = -1

Funciones. Texto

- substring(inico, final)
  - var mensaje = "Hola Mundo";
  - var porcion = mensaje.substring(2); // porcion = "la Mundo"
  - porcion = mensaje.substring(5); // porcion = "Mundo"
  - porcion = mensaje.substring(7); // porcion = "ndo"
  - porcion = mensaje.substring(-2); // porcion = "Hola Mundo"
  - porcion = mensaje.substring(1, 8); // porcion = "ola Mun"
  - porcion = mensaje.substring(3, 4); // porcion = "a"
  - porcion = mensaje.substring(5, 0); // porcion = "Hola "
- split(separador)
  - var mensaje = "Hola Mundo, soy una cadena de texto!";
  - var palabras = mensaje.split(" ");
  - // palabras = ["Hola", "Mundo,", "soy", "una", "cadena", "de", "texto!"];
  - var palabra = "Hola";
  - var letras = palabra.split(""); //letras = ["H", "o", "l", "a"]

Funciones y Propiedades para Números

- Eval: Permite evaluar matemáticamente una cadena

```
var miTexto = "3 + 5"
var valor= eval(miTexto )
```

- ParseInt, ParseFloat , … Convierte una cadena a número
- IsNaN(): devuelve un boleano si lo recibido es un número o no

```
var numero1 = 0;
var numero2 = 0;
if(isNaN(numero1/numero2)) {
  alert("La división no está definida para los números indicados");
} else {
  alert("La división es igual a => " + numero1/numero2);
}
```

- toFixed(digitos): var numero1 = 4564.34567;
  - numero1.toFixed(2); // 4564.35
  - numero1.toFixed(6); // 4564.345670
  - numero1.toFixed(); // 4564

Funciones para Arrays

- concat()
  - var array1 = [1, 2, 3];
  - array2 = array1.concat(4, 5, 6); // array2 = [1, 2, 3, 4, 5, 6]
  - array3 = array1.concat([4, 5, 6]); // array3 = [1, 2, 3, 4, 5, 6]
- join(separador)
  - var array = ["hola", "mundo"];
  - var mensaje = array.join(""); // mensaje = "holamundo"
  - mensaje = array.join(" "); // mensaje = "hola mundo"

Funciones para Arrays

- pop(), elimina el último elemento del array y lo devuelve. El array original se modifica y su longitud disminuye en 1 elemento.
  - var array = [1, 2, 3];
  - var ultimo = array.pop();
  - // ahora array = [1, 2], ultimo = 3
- push(), añade un elemento al final del array. El array original se modifica y aumenta su longitud en 1 elemento.
  - var array = [1, 2, 3];
  - array.push(4);
  - // ahora array = [1, 2, 3, 4]

Funciones para Arrays

- shift(), elimina el primer elemento del array y lo devuelve. El array se ve modificado y su longitud se reduce en 1 elemento.
  - var array = [1, 2, 3];
  - var primero = array.shift();
  - // ahora array = [2, 3], primero = 1
- unshift(), añade un elemento al principio del array. El array original se modifica y aumenta su longitud en 1 elemento.
  - var array = [1, 2, 3];
  - array.unshift(0);
  - // ahora array = [0, 1, 2, 3]
- Reverse():
  - var array = [1, 2, 3];
  - array.reverse();
  - // ahora array = [3, 2, 1]

Funciones de Fecha

Metodos de Date()	Funcionalidad
getDate()	Devuelve el día del mes.
getDay()	Devuelve el día de la semana
getHours()	Retorna la hora.
getMinutes()	Devuelve los minutos.
getMonth()	Devuelve el mes (atención al mes que empieza por 0)
getSeconds()	vuelve los segundos.
setDate()	Actualiza el día del mes.
setHours()	Actualiza la hora.
setMinutes()	Cambia los minutos.
setMonth()	Cambia el mes (atención al mes que empieza por 0)

Funciones de Fecha

```
<HTML>
<HEAD> <TITLE>Ejemplo que muestra la fecha y hora del sistema</TITLE></HEAD>
<SCRIPT TYPE="text/javascript">
<!--
function muestraFecha() {
var d = new Date();
document.write("Fecha completa:"+ Date() + "<BR>");
document.write("Día del mes:"+ d.getDate() + "<BR>");
document.write("Día de la semana:"+ d.getDay() + "<BR>");
<!--0=Domingo a 6=Sábado -->
document.write("Hora:"+ d.getHours() + "<BR>");
document.write("Minutos:"+ d.getMinutes() + "<BR>");
document.write("Segundos:"+ d.getSeconds() + "<BR>");
document.write("Mes:"+ d.getMonth() + "<BR>");
document.write("Año:"+ d.getYear() + "<BR>");
document.write("Hora completa:"+ d.getTime() + "<BR>");
}
// -->
</SCRIPT>
<BODY onload = muestraFecha()></BO DY></HTML>
```

Objetos en JavaScript

- Crear un nuevo objeto implica dotarlo de propiedades y métodos, es decir, crear una especie de clase de la que luego se puedan hacer instancias.
- Ha de tenerse en cuenta que los objetos no se pueden destruir, sino que perviven mientras esté el documento cargado.
- Para crear un objeto, tenemos que crear una función **constructora**. Por ejemplo, un clase casa sería:

```
function Casa( p_ciudad, p_plantas,
          p_habitaciones, p_banios ) {
  this.ciudad=p_ciudad;
  this.plantas=p_plantas;
  this.habitaciones=p_habitaciones;
  this.banios=p_banios;
} // function Casa
```

- Los objetos casa tiene propiedades/atributos y se crean con new

```
var casa1=new Casa( "Jaen", 1, 3, 2);
var casa2=new Casa( "Huelva", 2, 4, 3 );
casa2.patio="60m2"; //Se modifica el valor de la propiedad patio de Casa2
```

Palabras Reservadas

- El estándar ECMA-262 incluye la lista de las palabras reservadas que utiliza actualmente JavaScript y la lista de las palabras reservadas para su uso futuro.
- Utilizadas actualmente:
  - break, else, new, var, case, finally, return, void, catch, for, switch, while, continue, function, this, with, default, if, throw, delete, in, try, do, instanceof, typeof
- Reservadas para su uso futuro:
  - abstract, enum, int, short, boolean, export, interface, static, byte, extends, long, super, char, final, native, synchronized, class, float, package, throws, const, goto, private, transient, debugger, implements, protected, volatile, double, import, public

Indice

- Introducción
- DOM
- BOM
- Eventos
- Formularios

1. Validación

- JavaScript puede ser utilizado para validar la entrada de datos antes de enviarla al servidor.
- Los datos de formas que son verificados generalmente por JavaScript son:
  - ¿Se han dejando vacíos espacios requeridos?
  - ¿Ha puesto el usuario una dirección de correo electrónico válido?
  - ¿Ha puesto el usuario una fecha válida?
  - ¿Ha puesto el usuario texto en un campo numérico?
- Ejemplos:
  - http://www.w3schools.com/js/js_form_validation.asp

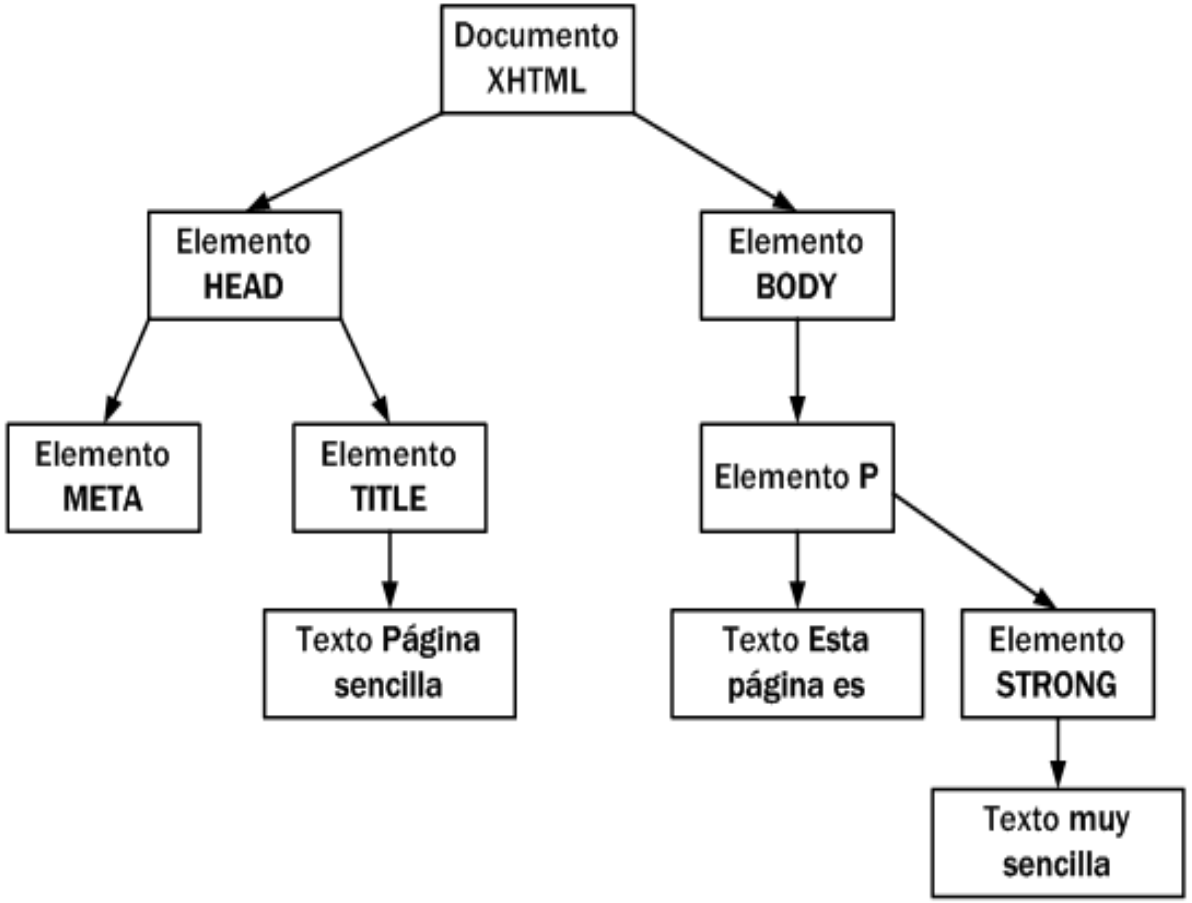
2. DOM

- DOM = Document Object Model
- DOM se diseñó originalmente para manipular de forma sencilla los documentos XML.
- De esta forma, permite acceder y manipular las páginas XHTML
- A pesar de sus orígenes, DOM se ha convertido en una utilidad disponible para la mayoría de lenguajes de programación (Java, PHP, JavaScript) y cuyas únicas diferencias se encuentran en la forma de implementarlo.
- El responsable del DOM es el World Wide Web Consortium (W3C).

2. DOM

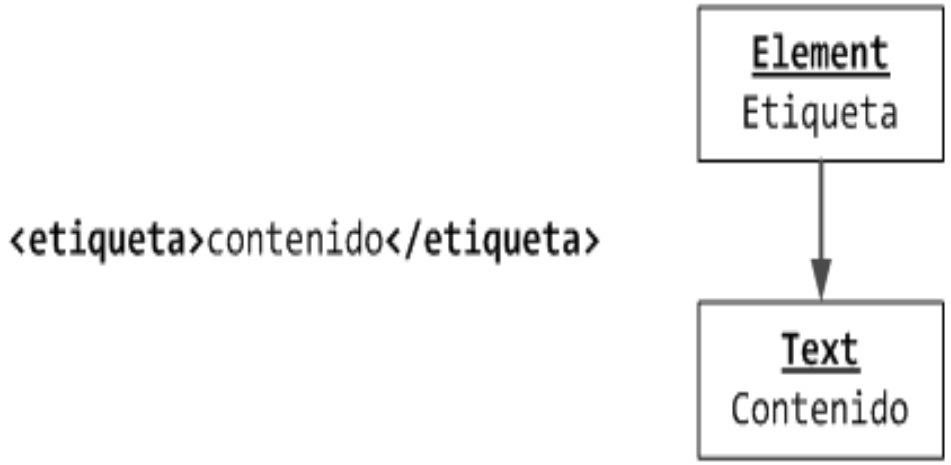
- DOM transforma los documentos XHTML en un conjunto de elementos llamados nodos.

```
<html
xmlns="http://www.w3.org/1999/
xhtml">
<head>
<title>Página sencilla</title>
</head>
<body>
<p>Esta página es <strong>muy
sencilla</strong></p>
</body>
</html
```



2. DOM

- Las etiquetas XHTML se transforman en dos nodos:
  - el primero es la propia etiqueta y
  - el segundo nodo es hijo del primero y consiste en el contenido textual de la etiqueta.
- Si una etiqueta XHTML se encuentra dentro de otra, se sigue el mismo procedimiento anterior, pero los nodos generados serán nodos hijo de su etiqueta padre.



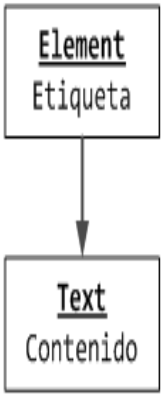
2. DOM

- La especificación completa de DOM define 12 tipos de nodos:
  - Document, nodo raíz del que derivan todos los demás nodos del árbol.
  - Element, representa cada una de las etiquetas XHTML. Se trata del único nodo que puede contener atributos y el único del que pueden derivar otros nodos.
  - Attr, se define un nodo de este tipo para representar cada uno de los atributos de las etiquetas XHTML, es decir, uno por cada par atributo=valor.
  - Text, nodo que contiene el texto encerrado por una etiqueta XHTML.
  - Comment, representa los comentarios incluidos en la página XHTML.
  - Los otros tipos de nodos existentes que no se van a considerar son DocumentType, CDataSection, DocumentFragment, Entity, EntityReference, ProcessingInstruction y Notation.

2. DOM

- Creación de nodos → Requiere 4 pasos:
  - Creación de un nodo tipo Element
  - Creación de un nodo tipo Text
  - Añadir el nodo Text como nodo hijo del nodo Element
  - Añadir el nodo Element a la página, en forma de nodo hijo del nodo correspondiente donde queremos insertar el elemento

<etiqueta>contenido</etiqueta>




```
// Crear nodo de tipo Element
var parrafo = document.createElement("p");

// Crear nodo de tipo Text
var contenido = document.createTextNode("Hola
Mundo!");

// Añadir el nodo Text como hijo del nodo Element
parrafo.appendChild(contenido);

// Añadir el nodo Element como hijo de la pagina
document.body.appendChild(parrafo);
```

- Eliminación de nodos → RemoveChild():
  - Elimina un nodo de un documento XHTML
  - Requiere como parámetro el nodo que se va a eliminar (se tiene que haber seleccionado previamente).
  - Esta función debe ser invocada desde el elemento padre de ese nodo que se quiere eliminar.
  - La forma más segura y rápida de acceder al nodo padre de un elemento es mediante la propiedad nodoHijo.parentNode.

```
var parrafo = document.getElementById("provisional");
parrafo.parentNode.removeChild(parrafo);
```

```
<p id="provisional">...</p>
```

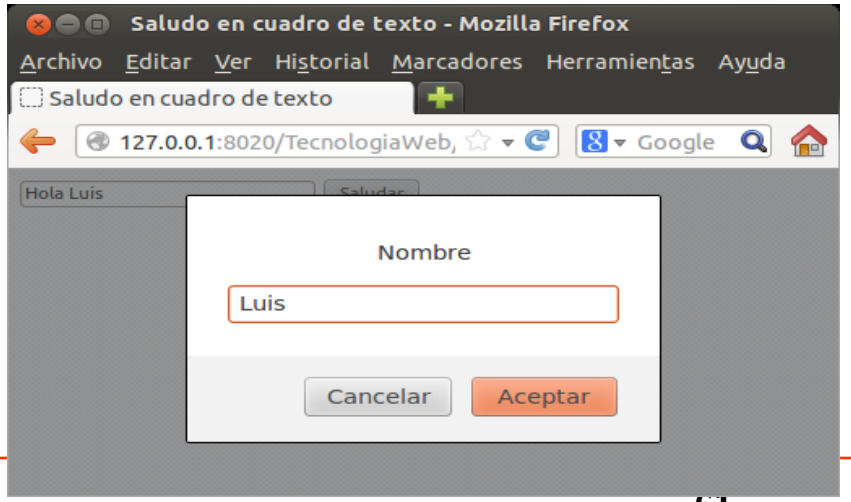
2. DOM

- Hay dos maneras de acceder a los nodos:
  - **A través del nodo padre**: consisten en acceder al nodo raíz de la página y después a sus nodos hijos y a los nodos hijos de esos hijos y así sucesivamente hasta el último nodo de la rama terminada por el nodo buscado.
  - **De forma directa**: Es mucho más rápido acceder directamente a ese nodo y no llegar hasta él descendiendo a través de todos sus nodos padre.
- Solo se puede acceder a los nodos cuando el árbol DOM ha sido construido completamente, es decir, si el documento XHTML no es válido → No se puede acceder

2. DOM

- Hay dos maneras de acceder a los nodos:
  - **A través del nodo padre (Ejercicio1)**:

```
<html xmlns="http://www.w3.org/1999/xhtml">
<head>
<title>Hola</title>
<script type="text/javascript">
function Saludar() {
  var nombre;
  nombre = window.prompt("Nombre");
  window.document.formulario.texto.value = "Hola " + nombre;
}
</script>
</head>
<body>
  <form id="formulario" name="formulario">
    <input type="text" name="text" id="texto"/>
    <input type="button" onclick="Saludar()"
          value="Saludar"/>
  </form>
</body>
</html>
```



2. DOM

- Sin embargo la manera más sencilla de hacerlo a través de su nombre o identificador (propiedades name e id de HTML) → Acceso directo
  - Se accede por el nombre de la propiedad o por su atributo name o id
  - Algunos navegadores pueden no soportar el atributo id; sin embargo la nueva especificación de HTML recomienda utilizarlo en lugar de name
  - Para evitar problemas de compatibilidad, lo mejor es especificar los dos con el mismo valor
- Se utilizan los siguientes métodos para acceder a los nodos:
  - getElementsByTagName(etiqueta)
  - getElementsByName(nombre)
  - getElementById(id)

2. DOM

- Acceso Directo
  - getElementsByTagName(etiqueta): obtiene todos los elementos de la página XHTML cuya etiqueta sea igual que el parámetro que se le pasa a la función
  - getElementsByName(nombre): obtiene todos los elementos de la página XHTML cuya atributo nombre sea igual que el parámetro que se le pasa a la función
  - getElementById(id): obtiene el elemento de la página XHTML cuya atributo id sea igual que el parámetro que se le pasa a la función

- Acceso Directo
  - getElementsByTagName(etiqueta):

```
var parrafos = document.getElementsByTagName("p");
```

    - ✓ Devuelve **todos** los párrafos del documento XHTML como un array de objeto
      - parrafos[0]: Devolvería el contenido del 1º elemento <p> del documento
      - parrafos[1]: Devolvería el contenido del 2º elemento <p> del documento
      - ….
    - ✓ Para recorrer todos los párrafos se haría

```
for (var i = 0; i < parrafos.length; i++) {
        var parrafo = parrafos[i];
}
```

2. DOM

- Ejemplo
  - Párrafos: document.getElementsByTagName("p")
  - Enlace: document.links[0]
  - Imagen: document.images[0] o document.images["logotipo"]
  - Formulario: document.forms[0] o document.forms["consultas"]
  - Acceso al texto del text: document.forms.texto.value

```
<html>
  <head><title>Pagina de ejemplo</title></head>
  <body>
    <p>Primer parrafo de la pagina</p>
    <p>Segundo parrafo de la pagina</p>
    <a href="otra_pagina.html">Un enlace</a>
    <img src="logo.jpg" name="logotipo"/>
    <form method="post" name="consultas">
      <input type="text" name="id" />
      <input type="submit" value="Enviar">
    </form>
  </body>
</html>
```

2. DOM

- Mediante DOM, es posible acceder también a todos los atributos XHTML y todas las propiedades CSS
- Los atributos XHTML de los elementos de la página se transforma automáticamente en propiedades de los nodos
- Para obtener el valor de cualquier propiedad CSS del nodo, se debe utilizar el atributo style.

```
var imagen = document.getElementById("imagen");
alert(imagen.style.margin);

<img id = "imagen" style = "margin:0; border:0;" src="logo.png" />
```



2. DOM

- Si el nombre de una propiedad CSS es compuesto, se modifica un poco:
  - font-weight será fontWeight
  - line-height será lineHeight
  - border-top-style será borderTopStyle
  - list-style-image será listStyleImage

```
var parrafo = document.getElementById("parrafo");
alert(parrafo.style.fontWeight);  // muestra "bold"
```

- El único atributo XHTML que NO es igual en XHTML y en DOM es class → Para obtener la class se usa className

```
var parrafo = document.getElementById("parrafo");
alert(parrafo.class); // muestra "undefined"
alert(parrafo.className); // muestra "normal"
```

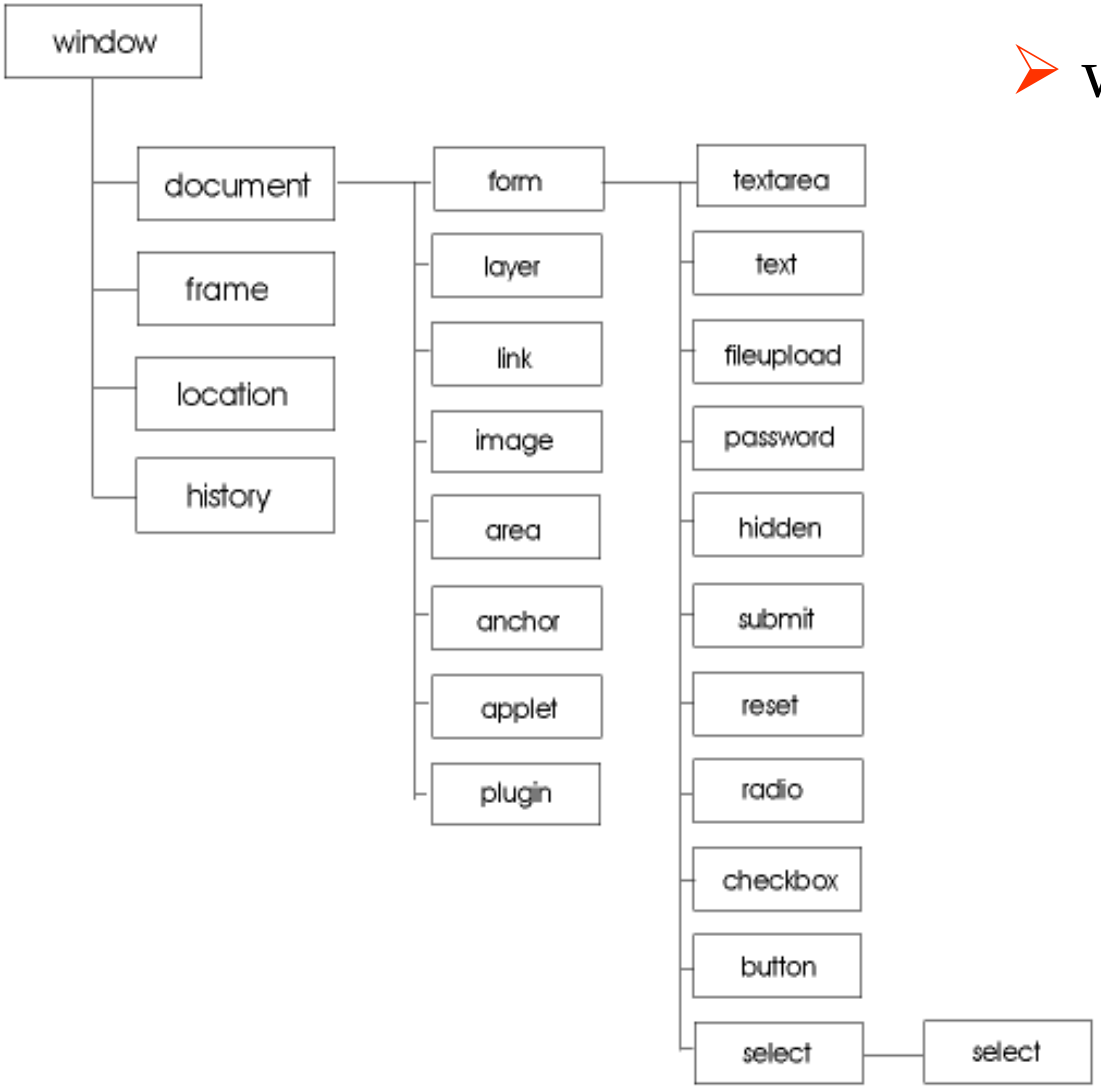
3. BOM: Objetos del navegador

- Las versiones 3.0 de los navegadores Internet Explorer y Netscape Navigator introdujeron el concepto de Browser Object Model o BOM, que permite acceder y modificar las propiedades de las ventanas del propio navegador.
- Algunos objetos siempre están presentes:
  - window (clase Window) ventana del navegador
  - document (clase Document) página cargada en la ventana
- Otros se crean según se van encontrando sus elementos correspondientes en el HTML
- Algunos objetos tienen propiedades que son otros objetos o colecciones de objetos
  - Todos los objetos del navegador están contenidos en propiedades de window, o en propiedades de objetos contenidos en window (recursivamente)
  - Se puede hablar de una jerarquía de objetos del navegador

3. BOM: Objetos del navegador

- El mayor inconveniente de BOM es que, al contrario de lo que sucede con DOM, ninguna entidad se encarga de estandarizarlo o definir unos mínimos de interoperabilidad entre navegadores.
- Algunos de los elementos que forman el BOM son los siguientes:
  - Crear, mover, redimensionar y cerrar ventanas de navegador.
  - Obtener información sobre el propio navegador.
  - Propiedades de la página actual y de la pantalla del usuario.
  - Gestión de cookies.
  - Objetos ActiveX en Internet Explorer.

3. BOM: Objetos del navegador



- window
  - history
  - location
  - document <body>...</body>
    - anchor ...
    - applet <applet>...</applet>
    - area <map>...</map>
    - form <form>...</form>
    - image
    - link ..
    - plugin <embed src="..." />
  - frame

3. BOM: Objetos del navegador

- Objeto History
  - Se encarga de almacenar una lista con los sitios por los que se ha estado navegando.
  - Se usa principalmente para movernos hacia atrás y adelante en dicha lista
  - Uso: history.propiedad
    - ✓ current – contiene URL completa de la entrada actual del historial
    - ✓ next – cadena del siguiente URL
    - ✓ length – Entero que contiene el número de entradas del historial
    - ✓ previous – cadena de la URL anterior
  - Métodos
    - ✓ back() - Vuelve a cargar la URL anterior
    - ✓ forward() - Vuelve a cargar la URL siguiente
    - ✓ go(posicion) – Carga el URL especificado por posicion, si posicion es entero indica la posición relativa dentro del historial, si posicion es una cadena indica la URL que esta en el historial

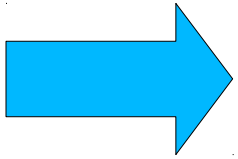
3. BOM: Objetos del navegador

- Objeto location
  - Contiene la URL actual así como algunos datos de interés respecto a esta URL.
  - Su finalidad principal es, por una parte, modificar el objeto location para cambiar a una nueva URL, y extraer los componentes de dicha URL de forma separada.
    - ✓ protocolo://maquina_host[:puerto]/camino_al_recurso
    - ✓ http://www.w3schools.com/js/js_window_location.asp

3. BOM: Objetos del navegador

- Objeto location: Propiedades:
  - ✓ hash – Cadena que contiene el nombre del enlace, dentro de la URL.
  - ✓ host - Cadena que contiene el nombre del servidor y el número del puerto, dentro de la URL.
  - ✓ hostname - Cadena que contiene el nombre de dominio del servidor (o la dirección IP)
  - ✓ href - Cadena que contiene la URL completa.
  - ✓ pathname - Cadena que contiene el camino al recurso.
  - ✓ port - Cadena que contiene el número de puerto del servidor.
  - ✓ protocol - Cadena que contiene el protocolo utilizado (incluyendo los dos puntos)
  - ✓ search - Cadena que contiene la información pasada en una llamada a un script

```
<script>
document.write(location.href);
document.write(location.pathname);
</script>
```



http://www.w3schools.com/js/js_window_location.asp
/js/js_window_location.asp

3. BOM: Objetos del navegador

- Objeto location: Métodos
  - reload() - Vuelve a cargar la URL especificada en la propiedad href del objeto location.
  - replace(cadenaURL) - Reemplaza el historial actual mientras carga la URL especificada en cadenaURL.
  - Assign(): Método que permite cargar una nueva página

```
<script>
        function newDoc()  {
                window.location.assign("http://www.w3schools.com")
        }
</script>
….
<input type="button" value="Load new document" onclick="newDoc()">
```

3. BOM: Objetos del navegador

- Objeto Screen
  - Ofrece información acerca del monitor donde se está ejecutando el navegador.
  - La propiedades principales del objeto screen son:
    - ✓ availHeight : El alto de la pantalla en pixeles disponible para el navegador.
    - ✓ availWidth : El ancho de la pantalla en pixeles disponible para el navegador.
    - ✓ colorDepth : Representa el número de bits usados para representar los colores.
    - ✓ height : El alto de la pantalla en pixeles.
    - ✓ width : El ancho de la pantalla en pixeles.

```
<head>
<title>Problema</title>
</head>
<body>
<script type="text/javascript">
  document.write('Valores de las propiedades del objeto screen:<br>');
  document.write('availHeight :' + screen.availHeight + '<br>');
  document.write('availWidth :' + screen.availWidth + '<br>');
  document.write('height :' + screen.height + '<br>');
  document.write('width :' + screen.width + '<br>');
  document.write('colorDepth :' + screen.colorDepth);
</script>
</body>
```

3. BOM: Objetos del navegador

- Objeto Navigation
  - Este objeto nos da la información relativa al navegador que esté utilizando el usuario
  - Propiedades:
    - ✓ appCodeName - Nombre del código del cliente.
    - ✓ appName – Nombre del cliente
    - ✓ appVersion - Información sobre la versión del cliente.
    - ✓ language - información sobre el idioma de la versión del cliente
    - ✓ mimeTypes - Array que contiene todos los tipos MIME soportados por el cliente.
    - ✓ platform - plataforma sobre la que se está ejecutando el programa cliente
    - ✓ plugins - Array que contiene todos los plug-ins soportados por el cliente
    - ✓ UserAgent - Cabecera completa del agente enviada en una petición HTTP.

3. BOM: Objetos del navegador

- Objeto Navigation
  - Este objeto nos da la información relativa al navegador que esté utilizando el usuario
  - Métodos
    - javaEnabled() - Devuelve true si el cliente permite la utilización de Java, en caso contrario, devuelve false

3. BOM: Objetos del navegador

- Objeto document
  - Cada documento HTML cargado en el navegador se convierte en un objeto document (BODY <=> DOCUMENT)
  - Provee acceso a todos los elementos en una página HTML/XHTML donde se ejecuta el script
  - El objeto document también forma parte del objeto window y puede ser accesado a través de window.document

3. BOM: Objetos del navegador

- Objeto document: Propiedades
  - cookie – Establece o regresa todas las cookies asociadas al documento actual
  - domain – Regresa el nombre del dominio
  - lastModified – Fecha y hora de la última modificación del documento
  - title – Título del documento actual
  - URL – URL del documento actual

3. BOM: Objetos del navegador

- Objeto document: Métodos
  - getElementById(id) – Regresa una referencia del primer objeto con id especificado
  - getElementsByName(name) – Regresa una colección de objetos en el nombre especificado por name
  - getElementsByTagName(tag) – Regresa una colección de objetos con tag especificado.
  - write(cadena) – Escribe la cadena en el documento
  - writeln(cadena) – Idéntico al write() solo que agrega un caracter de salto del línea al final

3. BOM: Objetos del navegador

- Objeto document: Colecciones
  - anchors[] - Referencia a todas los objetos anclas en un document
  - forms[] - Referencia a todos los objetos de un formulario
  - images[] - Referencia a todos los objetos de imagen
  - links[] - Referencia a todas las areas y links del documento

3. BOM: Objetos del navegador

```
<HTML>
<BODY>
<A HREF="enlace1.htm">a</A>
<A HREF="enlace2.htm">b</A><hr>
<IMG SRC="images/boton1.png"><IMG SRC="images/boton2.png"><hr>
<SCRIPT TYPE="text/javascript">
<!--
for (i=2; i<100; i*=2){
document.write(i+"<br>");
}
document.write("<H3>Enlaces en el documento</H3>");
for(i=0;i<document.links.length;i++){
document.write(document.links[i],"<br>");
}
document.write("<H3>Imágenes en el documento</H3>");
for(i=0;i<document.images.length;i++){
document.write(document.images[i].src,"<br>");
}
// -->
</SCRIPT></BODY>
</HT ML>
```

3. BOM: Objetos del navegador

- Objeto window
  - Representa una ventana abierta en el navegador.
  - Mediante este objeto, es posible mover, redimensionar y manipular la ventana actual del navegador. Incluso es posible abrir y cerrar nuevas ventanas de navegador.
  - El objeto window apunta a la ventana actual
  - Si un documento contiene frames, el navegador crea un objeto window para el documento HTML y un objeto window adicional para cada frame
  - Colecciones
    - ✓ frames[] - Contiene una referencia a todos los frames en la ventana

3. BOM: Objetos del navegador

- Objeto window: Propiedades
  - closed - Regresa si una ventana ha sido o no cerrada
  - document – Referencia al objeto document
  - history – Referencia al objeto history
  - length – Número de frames en la ventana
  - location – Referencia al objeto location
  - name – Establece o regresa el nombre de la ventana
  - opener – Referencia a la ventana que creo la ventana actual
  - parent – Referencia al objeto window padre o que contiene el frameset
  - self – Referencia a la ventaja actual
  - status – Establece texto en la barra de estado
  - top – Regresa la ventana en el nivel más superior

3. BOM: Objetos del navegador

- Objeto window: Métodos
  - alert(mensaje) – Muestra mensaje de alerta
  - blur() - Elimina el foco de la ventana
  - clearInterval(id) – Elimina el intervalo id establecido por setInterval()
  - clearTimeout(name) - Elimina el intervalo name establecido por setTimeout()
  - close() - Cierra la ventana actual
  - confirm(mensaje) – Despliega un cuadro de diálogo con el mensaje y los botones 'Aceptar' y 'Cancelar'
  - focus() - Establece el foco a la ventana actual
  - moveBy(x, y) – Mueve la ventana el numero de pixeles especificados
  - moveTo(x, y) – Mueve la ventana a las coordenadas especificadas

3. BOM: Objetos del navegador

- Objeto window: Métodos
  - open(URL, nombre, opciones) – Abre una ventana
  - prompt(mensaje) – Muestra una caja de usuario que sugiere al usuario ingresar datos
  - setInterval(expresion, tiempo) - Evalua la expresión especificada después de que hayan pasado el número de milisegundos especificados en tiempo. Devuelve un valor que puede ser usado como identificativo por clearInterval()
  - setTimeout(expresion, tiempo) - Evalua la expresión especificada después de que hayan pasado el número de milisegundos especificados en tiempo. Devuelve un valor que puede ser usado como identificativo por clearTimeout()

```
setTimeout("alert('Han transcurrido 3 segundos
        desde que me programaron')", 3000);
```

3. BOM: Objetos del navegador

- BOM define cuatro métodos para manipular el tamaño y la posición de la ventana:
  - moveBy(x, y) desplaza la posición de la ventana x píxel hacia la derecha y y píxel hacia abajo. Se permiten desplazamientos negativos .
  - moveTo(x, y) desplaza la ventana del navegador hasta que la esquina superior izquierda se encuentre en la posición (x, y) de la pantalla del usuario. Se permiten desplazamientos negativos.
  - resizeBy(x, y) redimensiona la ventana del navegador de forma que su nueva anchura sea igual a (anchura_anterior + x) y su nueva altura sea igual a (altura_anterior + y). Se pueden emplear valores negativos.
  - resizeTo(x, y) redimensiona la ventana del navegador hasta que su anchura sea igual a x y su altura sea igual a y. No se permiten valores negativos.

```
// Mover la ventana 20 píxel hacia la derecha y 30 píxel hacia abajo
window.moveBy(20, 30);
// Redimensionar la ventana hasta un tamaño de 250 x 250
window.resizeTo(250, 250);
// Agrandar la altura de la ventana en 50 píxel
window.resizeBy(0, 50);
// Colocar la ventana en la esquina izquierda superior de la ventana
window.moveTo(0, 0);
```

3. BOM: Objetos del navegador

- Abriendo ventanas
  - URL – Opcional. URL de la página a abrir. Si no se especifíca se abre una about:blank
  - nombre – Opcional. Especifica el atributo target o el nombre de la ventana. Soporta:
    - ✓ _blank
    - ✓ _parent
    - ✓ _self
    - ✓ _top
    - ✓ Nombre
  -

3. BOM: Objetos del navegador

- Abriendo ventanas
  - Opciones: Opcional. Lista de elementos separados por coma
    - directories = yes|no|1|0
    - fullscreen = yes|no|1|0
    - height = pixeles
    - left = pixeles
    - location = yes|no|1|0
    - menubar = yes|no|1|0
    - resizable = yes|no|1|0
    - scrollbars = yes|no|1|0
    - status = yes|no|1|0
    - titlebar = yes|no|1|0
    - toolbar = yes|no|1|0
    - top = pixeles
    - width = pixeles

```
ventana = window.open("",
"", "width=800,
height=300, location=no,
directories=no,
menubar=no,
resizable=no");
```

4. Eventos

- Los eventos son acciones que pueden ser detectadas por JavaScript. Cada elemento de una página web tiene ciertos eventos que pueden llamar una función de JavaScript.
  - El clic del Mouse
  - Una página web o imagen cargándose
  - Poner el cursor del Mouse sobre algún lugar de la página.
  - Seleccionar una caja de entrada en una forma HTML.
  - Un teclazo.
- Historia
  - El nivel 1 de DOM no incluye especificaciones relativas a los eventos JavaScript.
  - El nivel 2 de DOM incluye ciertos aspectos relacionados con los eventos
  - El nivel 3 de DOM incluye la especificación completa de los eventos de JavaScript

4. Eventos

- La lista completa de eventos que se pueden generar en un navegador se puede dividir en cuatro grandes grupos. La especificación de DOM define los siguientes grupos:
  - Eventos de ratón: se originan cuando el usuario emplea el ratón para realizar algunas acciones.
  - Eventos de teclado: se originan cuando el usuario pulsa sobre cualquier tecla de su teclado.
  - Eventos HTML: se originan cuando se producen cambios en la ventana del navegador o cuando se producen ciertas interacciones entre el cliente y el servidor.
  - Eventos DOM: se originan cuando se produce un cambio en la estructura DOM de la página. También se denominan "eventos de mutación".

4. Eventos

- Eventos de ratón: Suelen ser los más empleados en las aplicaciones web.

Evento	Descripción
click	Se produce cuando se pulsa el botón izquierdo del ratón. También se produce cuando el foco de la aplicación está situado en un botón y se pulsa la tecla`ENTER`
dblclick	Se produce cuando se pulsa dos veces el botón izquierdo del ratón
mousedown	Se produce cuando se pulsa cualquier botón del ratón
mouseout	Se produce cuando el puntero del ratón se encuentra en el interior de un elemento y el usuario mueve el puntero a un lugar fuera de ese elemento
mouseover	Se produce cuando el puntero del ratón se encuentra fuera de un elemento y el usuario mueve el puntero hacia un lugar en el interior del elemento
mouseup	Se produce cuando se suelta cualquier botón del ratón que haya sido pulsado
mousemove	Se produce (de forma continua) cuando el puntero del ratón se encuentra sobre un elemento

4. Eventos

Eventos de teclado

Evento	Descripción
keydown	Se produce cuando se pulsa cualquier tecla del teclado. También se produce de forma continua si se mantiene pulsada la tecla
keypress	Se produce cuando se pulsa una tecla correspondiente a un carácter alfanumérico (no se tienen en cuenta telas como `SHIFT`, `ALT`, etc.). También se produce de forma continua si se mantiene pulsada la tecla
keyup	Se produce cuando se suelta cualquier tecla pulsada

- Objeto event
  - La propiedad keyCode
  - La propiedad charCode (sólo DOM)
  - La propiedad srcElement (Internet Explorer) o target (DOM)
  - Las propiedades shiftKey, ctrlKey, altKey y metaKey (sólo DOM)

4. Eventos

Eventos de HTML

Evento	Descripción
load	Se produce en el objeto `window` cuando la página se carga por completo. En el `<img>` cuando se carga por completo la imagen. En el elemento`<object>` cuando se carga el objeto
unload	Se produce en el objeto `window` cuando la página desaparece por completo (al cerrar la ventana del navegador por ejemplo). En el elemento `<object>` cuando desaparece el objeto.
abort	Se produce en un elemento `<object>` cuando el usuario detiene la descarga del elemento antes de que haya terminado
error	Se produce en el objeto `window` cuando se produce un error . En el `<img>` cuando la imagen no se ha podido cargar por completo y en `<object>` cuando el elemento no se carga correctamente
select	Se produce cuando se seleccionan varios caracteres de un cuadro de texto (`input` y `textarea`)
change	Se produce cuando un cuadro de texto (`<input>` y `<textarea>`) pierde el foco y su contenido ha variado. También se produce cuando varía el valor de un elemento`<select>`
submit	Se produce cuando se pulsa sobre un botón de tipo submit (`<input type="submit">`)
reset	Se produce cuando se pulsa sobre un botón de tipo reset (`<input type="reset">`)
resize	Se produce en el objeto `window` cuando se redimensiona la ventana del navegador
scroll	Se produce en cualquier elemento que tenga una barra de scroll, cuando el usuario la utiliza. El elemento `<body>` contiene la barra de scroll de la página completa
focus	Se produce en cualquier elemento (incluido el objeto `<body>`) cuando el elemento obtiene el foco

4. Eventos

- Eventos DOM
  - Estos eventos aún no han sido implementados en todos los navegadores.

Evento	Descripción
`DOMSubtreeModified`	Se produce cuando se añaden o eliminan nodos en el subárbol de un documento o elemento
`DOMNodeInserted`	Se produce cuando se añade un nodo como hijo de otro nodo
`DOMNodeRemoved`	Se produce cuando se elimina un nodo que es hijo de otro nodo
`DOMNodeRemovedFromDocument`	Se produce cuando se elimina un nodo del documento
`DOMNodeInsertedIntoDocument`	Se produce cuando se añade un nodo al documento

4. Eventos

- Manejadores de eventos
  - Los eventos deben asociarse a funciones o código JavaScript
  - Cuando se produce un evento se ejecuta el código indicado, por lo que la aplicación puede responder ante cualquier evento que se produzca durante su ejecución.
  - Las funciones o código JavaScript que se definen para cada evento se denominan "manejador de eventos", existiendovarias formas diferentes de indicar los manejadores:
    - ✓ Manejadores como atributos de los elementos XHTML.
    - ✓ Manejadores como funciones JavaScript externas.
    - ✓ Manejadores "semánticos".

4. Eventos

- Manejadores de eventos
  - Manejadores como atributos de los elementos XHTML.
    - ✓ Manejadores de eventos como atributos XHTML
    - ✓ Manejadores de eventos y variable this
  - El código se incluye en un atributo del propio elemento XHTML.

```
<body>
   <input type="button" value="Pinchame y verás"
                onclick="alert('Gracias por pinchar');" />
</body>
```

```
<body onload="alert('Página Cargada');">
```

4. Eventos

- Manejadores de eventos
  - Manejadores como atributos de los elementos XHTML.
    - ✓ En ocasiones puede ser muy difícil incluir el código Javascript como atributo y se utiliza la variable this
  - Ejemplo
    - ✓ Cuando el ratón pase por encima del , el color del borde cambiaa negro. Al salir, vuelve al color gris claro original

Div Original

```
<div id="contenidos" style="width:150px; height:60px; border:thin solid silver">
  Sección de contenidos...
</div>
```

Div con eventos

```
<div id="contenidos" style="width:150px; height:60px; border:thin solid silver"
onmouseover="document.getElementById('contenidos').style.borderColor='black';"
onmouseout="document.getElementById('contenidos').style.borderColor='silver';">
  Sección de contenidos...
</div>
```

Div con this

```
<div id="contenidos" style="width:150px; height:60px; border:thin solid silver"
onmouseover="this.style.borderColor='black';"
onmouseout="this.style.borderColor='silver';">
  Sección de contenidos...
</div>
```

4. Eventos

- Manejadores de eventos
  - Manejadores como funciones JavaScript externas.
    - ✓El método anterior se complica en exceso en cuanto se añaden varias instrucciones, por lo que no es recomendable
    - ✓Es aconsejable agrupar todo el código JavaScript en una función externa y llamar a esta función desde el elemento XHTML.

```
<input type="button" value="Pinchame y verás" onclick="muestraMensaje()" />

function muestraMensaje() {
  alert('Gracias por pinchar');
}
```

4. Eventos

- Manejadores de eventos
  - Manejadores como funciones JavaScript externas.
    - El principal inconveniente de este método es que en las funciones externas no se puede seguir utilizando la variable this y por tanto, es necesario pasar esta variable como parámetro a la función:

```
function resalta(elemento) {
  switch(elemento.style.borderColor) {
    case 'silver':
    case 'silver silver silver silver':
    case '#c0c0c0':
      elemento.style.borderColor = 'black';
      break;
    case 'black':
    case 'black black black black':
    case '#000000':
      elemento.style.borderColor = 'silver';
      break;
  }
}
```

```
<div style="width:150px; height:60px; border:thin solid silver"
onmouseover="resalta(this)" onmouseout="resalta(this)">
  Sección de contenidos...
</div>
```

4. Eventos

- Manejadores de eventos
  - Manejadores "semánticos"
    - ✓ Las soluciones anteriores (como atributos XHTML y como funciones externas) tienen el inconveniente de incluir mucho código en el código XHTML de la página.
    - ✓ Esta técnica es una evolución del método de las funciones externas, ya que se basa en utilizar las propiedades DOM de los elementos XHTML para asignar todas las funciones externas que actúan de manejadores de eventos.

4. Eventos

- Manejadores de eventos
  - Manejadores "semánticos": Consiste en
    - ✓ Asignar un identificador único al elemento XHTML mediante el atributo id.
    - ✓ Crear una función de JavaScript encargada de manejar el evento.
    - ✓ Asignar la función externa al evento correspondiente en el elemento deseado.
    - ✓ CUIDADO: No poner los paréntesis en muestraMensaje pues se ejecutaría en lugar de definir

```
// Función externa
function muestraMensaje() {
  alert('Gracias por pinchar');
}

// Asignar la función externa al elemento
document.getElementById("pinchable").onclick = muestraMensaje;

// Elemento XHTML
<input id="pinchable" type="button" value="Pinchame y verás" />
```

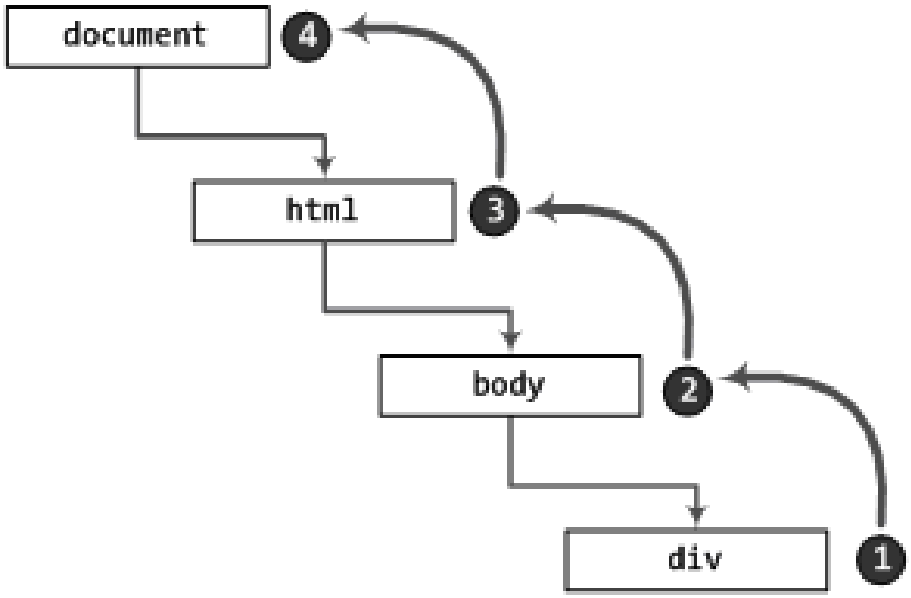
4. Eventos

- El flujo de eventos permite que varios elementos diferentes puedan responder a un mismo evento.
- El orden en el que se ejecutan los eventos asignados a cada elemento de la página es lo que constituye el flujo de eventos.

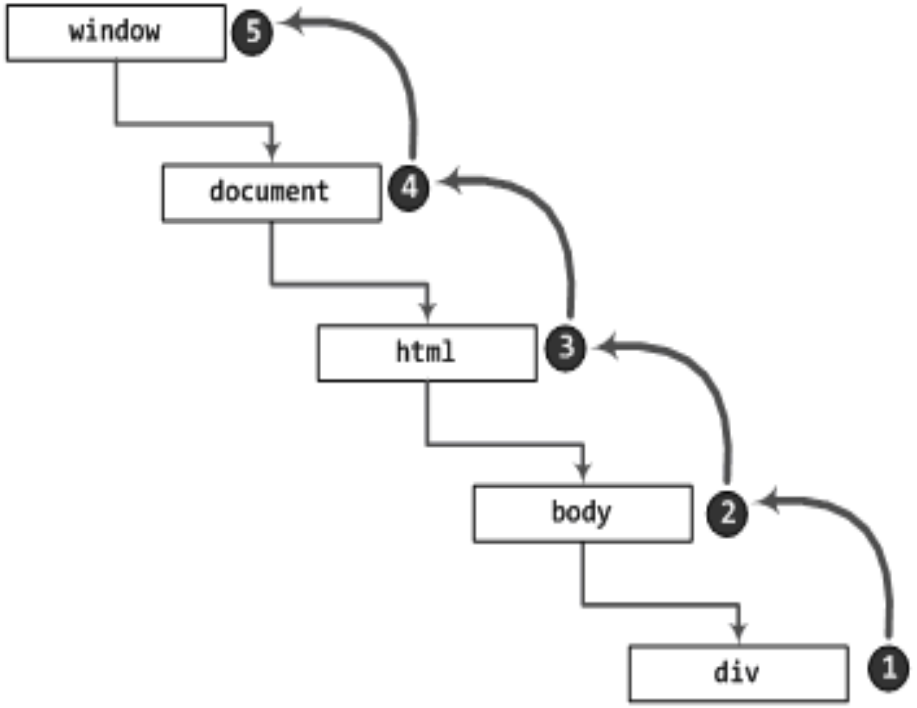
```
<html onclick="procesaEvento()">
<head>
        <title>Ejemplo de flujo de eventos</title>
</head>
  <body onclick="procesaEvento()">
        <div onclick="procesaEvento()">
                Pincha aqui
        </div>
  </body>
</html>
```

4. Eventos

- Cuando se pulsa sobre el texto "Pincha aquí" que se encuentra dentro del ,



IE



Firefox

4. Eventos

- El objeto Event
  - Cuando hay un evento, no es suficiente con asignarle una función responsable de procesar ese evento.
  - Es necesario obtener información adicional sobre el evento: la tecla que se ha pulsado, la posición del ratón, el elemento que ha producido el evento, etc.
  - El objeto event es el mecanismo definido por los navegadores para proporcionar toda esa información.
  - El estándar DOM especifica que el objeto event es el único parámetro que se debe pasar a las funciones encargadas de procesar los eventos.

4. Eventos

- La principal diferencia reside en la forma en la que se obtiene el objeto event.
  - Internet Explorer considera que este objeto forma parte del objeto window
  - El resto de navegadores lo consideran como el único argumento que tienen las funciones manejadoras de eventos.

IE

```
var evento = window.event;
```

Firefox

```
function manejadorEventos(elEvento) {
  var evento = elEvento;
}
```

Dos Soluciones

```
var evento = elEvento || window.event;
var myEvent = aEvent ? aEvent : window.event;
```

4. Eventos

- La propiedad type indica el tipo de evento producido,

var tipo = evento.type;

```
function resalta(elEvento) {
  var evento = elEvento || window.event;
  var myEvent = aEvent ? aEvent : window.event;
  switch(evento.type) {
    case 'mouseover':
      this.style.borderColor = 'black';
      break;
    case 'mouseout':
      this.style.borderColor = 'silver';
      break;
  }
}
window.onload = function() {
  document.getElementById("seccion").onmouseover = resalta;
  document.getElementById("seccion").onmouseout = resalta;
}

<div id="seccion" style="width:150px; height:60px; border:thin solid
silver">
  Sección de contenidos...
</div>
```

4. Eventos

- Información del evento
  - La propiedad type indica el tipo de evento producido,

    var tipo = evento.type;
  - Evento Raton
    - ✓ pageX, pageY
    - ✓ clientX, clientY
    - ✓ screenX, screenY
  - Evento teclado: (onkeypress)
    - ✓ keyCode // IE
    - ✓ which // Netscape/Firefox/Opera

4. Eventos

```
<HTML>
<HEAD>
<TITLE>Ejemplo sencillo que intercambia dos
imágenes</TITLE>
</HEAD>
<BODY>
<A HREF="http://www.google.es"
onMouseOver="boton.src='boton2.png'; return true"
onMouseOut="boton.src='boton1.png'; return true">
<IMG SRC="boton1.png" BORDER="0" NAME="boton">
</A>
</BODY>
</HT ML>
```

4. Eventos

- Conclusiones:
  - Una gran parte de la operabilidad de Javascript se ha trasladado a CSS
    - ✓ Cambiar el color de un botón cuando estas encima o cuando lo abandonas
    - ✓ Desarrollo de menú interactivos
    - ✓ Etc
  - El tratamiento de los formularios web siempre ha sido una de las tareas fundamentales de JavaScript.

5. Formularios

- Cuando se carga una página web, el navegador crea automáticamente un array llamado forms y que contiene la referencia a todos los formularios de la página.
  - document.forms[0]; //Primer formulario
- Además se crea automáticamente un array llamado elements por cada uno de los formularios de la página con todos sus elementos (cuadros de texto, botones, listas desplegables, etc.)
  - document.forms[0].elements[0];
  - document.forms[0].elements[document.forms[0].elements.length-1];
- Sin embargo, el mecanismo más habitual de acceso a los formularios es mediante el id o el name del formulario.
  - var formularioPrincipal = document.formulario;
  - var formularioSecundario = document.otro_formulario;
  - var formularioPrincipal = document.getElementById("formulario");
  - var primerElemento = document.getElementById("elemento");

5. Formularios

- Cada elemento dispone de las siguientes propiedades útiles para el desarrollo de las aplicaciones:
  - type: indica el tipo de elemento que se trata.
    - ✓ Para los elementos de tipo <input> (text, button, checkbox, etc.) coincide con el valor de su atributo type.
    - ✓ Para las listas desplegables (elemento <select>)
      - su valor es select-one y select-multiple.
  - form: es una referencia directa al formulario Así, para acceder al formulario de un elemento, se puede utilizar
    - ✓ document.getElementById("id_del_elemento").form
  - name: obtiene el valor del atributo name de XHTML. Solamente se puede leer su valor, por lo que no se puede modificar.
  - value: permite leer y modificar el valor del atributo value de XHTML.
    - ✓ (<input type="text"> y <textarea>) obtiene el texto que ha escrito el usuario.
    - ✓ Para los botones obtiene el texto que se muestra en el botón.

5. Formularios

- Obtener el valor de un cuadro de texto y textarea

```
<input type="text" id="texto" />
var valor = document.getElementById("texto").value;

<textarea id="parrafo"></textarea>
var valor = document.getElementById("parrafo").value;
```

- Obtener valor de un radioButton

```
<input type="radio" value="si" name="pregunta" id="pregunta_si"/> SI
<input type="radio" value="no" name="pregunta" id="pregunta_no"/> NO
<input type="radio" value="nsnc" name="pregunta" id="pregunta_nsnc"/> NS/NC


 var elementos = document.getElementsByName("pregunta");

 for(var i=0; i<elementos.length; i++) {
   alert(" Elemento: " + elementos[i].value + "\n Seleccionado: " +
 elementos[i].checked);
 }
```

Obtener valor de un checkbok

```
<input type="checkbox" value="condiciones" name="condiciones"
id="condiciones"/> He leído y acepto las condiciones
<input type="checkbox" value="privacidad" name="privacidad"
id="privacidad"/> He leído la política de privacidad
```

```
var elemento = document.getElementById("condiciones");
alert(" Elemento: " + elemento.value + "\n Seleccionado: " +
elemento.checked);
elemento = document.getElementById("privacidad");
alert(" Elemento: " + elemento.value + "\n Seleccionado: " +
elemento.checked)
```

5. Formularios

- Obtener valor de un select.
  - options, es un array para cada lista desplegable y que contiene la referencia a todas las opciones de esa lista.
    - ✓ document.getElementById("id_de_la_lista").options[0].
  - SelectedIndex devuelve la opción seleccionada

```
// Obtener el objeto select
var lista = document.getElementById("opciones");

// Obtener el valor de la opción seleccionada
var valorSeleccionado = lista.options[lista.selectedIndex].value;

// Obtener el texto que muestra la opción seleccionada
var valorSeleccionado = lista.options[lista.selectedIndex].text;
```

- Obtener valor de un select: Múltiple.

```
function multiple()
{
  var selectedArray = new Array();
  var selObj = document.getElementById('multiple');
  var i;
  var count = 0;
  for (i=0; i<selObj.options.length; i++) {
    if (selObj.options[i].selected) {
      selectedArray[count] = selObj.options[i].value;
      count++;
    }
  }
}
```

5. Formularios

- Evitar el envio duplicado
  - Una buena práctica en el diseño de aplicaciones web suele ser la de deshabilitar el botón de envío después de la primera pulsación.

```
<form id="formulario" action="#">
  ...
  <input type="button" value="Enviar"
         onclick="this.disabled=true;
         this.value='Enviando...';
         this.form.submit()" />
</form>
```

- Limitar el texto de un textarea
  - Imposibilidad de limitar el máximo número de caracteres que se pueden introducir, de forma similar al atributo maxlength de los cuadros de texto normales.

```
function limita(maximoCaracteres) {
  var elemento = document.getElementById("texto");
  if(elemento.value.length >= maximoCaracteres ) {
    return false;
  }
  else {
    return true;
  }
}


<textarea id="texto" onkeypress="return
limita(100);"></textarea>
```

5. Formularios

- Validación de un formulario
  - La principal utilidad de JavaScript en el manejo de los formularios es la validación de los datos introducidos por los usuarios.

```
<form action="" method="" id="" name="" onsubmit="return validacion()">
  ...
</form>

function validacion() {
  if (condicion que debe cumplir el primer campo del formulario) {
    // Si no se cumple la condicion...
    alert('[ERROR] El campo debe tener un valor de...');
    return false;
  }
  else if (condicion que debe cumplir el segundo campo del formulario)
{
    return false;
  }
  // Si el script ha llegado a este punto, es válido
  return true;
}
```

6. Librerías de JavaScript

- JQuery:
  - Es el framework más popular
  - Lo usan: Google, Microsoft, IBM, NetFlix
- Prototype
- MooTools
- YUI – Desarrollado por Yahoo
  - Tienes muchas funciones y ejemplo
- Ext JS: Para construir aplicaciones RIA
- Dojo -
- script.aculo.us
- UIZE

6. Librerías de Javascript

- Antes de comentar las diferentes librerías, es necesario presentar el concepto AJAX, pues lo emplean.
- AJAX (Ashyncronous JavaScript And XML).
  - Técnica de desarrollo Web que incorpora varias tecnologías, como son el JavaScript y XML, consiguiendo de esta manera una forma de navegar rápida, ágil y dinámica.
- Ajax es una tecnología asíncrona, en el sentido de que los datos adicionales se solicitan al servidor y se cargan en segundo plano sin interferir con la visualización ni el comportamiento de la página.
- JavaScript es el lenguaje interpretado (scripting language) en el que normalmente se efectúan las funciones de llamada de Ajax mientras que el acceso a los datos se realiza mediante XMLHttpRequest, objeto disponible en los navegadores actuales.

6. Librerías de Javascript

- Ajax es una combinación de cuatro tecnologías ya existentes:
  - XHTML (o HTML) y hojas de estilos en cascada (CSS) para el diseño que acompaña a la información.
  - Document Object Model (DOM) accedido con un lenguaje de scripting por parte del usuario.
  - El objeto XMLHttpRequest para intercambiar datos de forma asíncrona con el servidor web.
  - XML es el formato usado generalmente para la transferencia de datos solicitados al servidor, aunque cualquier formato puede funcionar, incluyendo HTML preformateado, texto plano, JSON y hasta EBML.

6. Librerías de Javascript

- Las ventajas de AJAX son
  - Mayor interactividad
  - Recuperación asíncrona de datos, reduciendo el tiempo de espera del usuario
  - Facilidad de manejo del usuario
  - El usuario tiene una mayor concienciaciño de las aplicaciones como aplicaciones de escritorio
  - Se reduce el tamaño de la información intercambiada
  - Portabilidad entre plataformas
  - No requieren instalación de complementos

6. Librerías de Javascript

- Las desventajas de AJAX son
  - Botón de volver atrás del navegador
  - Empleo de iframes ocultos para almacenar el historial
  - Empleo de fragmento identificador del URL (‘#’) y recuperación mediante JavaScript
  - Problema al agregar marcadores/favoritos en un momento determinado de la aplicación
  - Los motores de búsqueda no analizan JavaScript. La información en la página dinámica no se almacena en los registros del buscador.
  - Problemas al imprimir páginas renderizadas dinámicamente
  - Tiempos de respuesta entre la petición del usuario y la respuesta del servidor
  - Requiere que los usuarios tengan el JavaScript activado en el navegador
  - En el caso de Internet Explorer 6 y anteriores, que necesita tener activado el ActiveX

6. Librerías de JavaScript: jQuery

- jQuery es una biblioteca de JavaScript, creada inicialmente por John Resig, que permite:
  - Simplificar la manera de interactuar con los documentos HTML
  - Manipular el árbol DOM
  - Manejar eventos
  - Desarrollar animaciones y
  - Agregar interacción con la técnica AJAX a páginas web.
- jQuery, al igual que otras bibliotecas, ofrece una serie de funcionalidades basadas en JavaScript que de otra manera requerirían de mucho más código
- La característica principal de la biblioteca es que permite cambiar el contenido de una página web sin necesidad de recargarla, mediante la manipulación del árbol DOM y peticiones AJAX. Para ello utiliza las funciones $() o jQuery().

- Función $()
  - La función $() recibe como parámetro una expresión CSS o el nombre de una etiqueta HTML y devuelve todos los nodos (elementos) que concuerden con la expresión.

```
$("#tablaAlumnos"); // Devolverá el elemento con id="tablaAlumnos"
$(".activo");       // Devolverá una matriz de elementos con class="activo"
```

  - Una vez seleccionado, se realiza las operaciones

```
// Se elimina el estilo (con removeClass()) y se aplica uno
//nuevo (con addClass()) a todos los nodos con class="activo"
$(".activo").removeClass("activo").addClass("inactivo");
```

```
// Anima todos los componentes con class="activo"
$(".activo").slideToggle("slow");
```

6. Librerías de JavaScript: jQuery

- Función jQuery()
  - Antes de realizar cualquier acción en el documento con esta funciónjQuery(), el documento debe estar listo. Para ello se usa $(document).ready(); de esta forma:

6. Librerías de JavaScript: Prototype

- Prototype es un framework escrito en JavaScript que se orienta al desarrollo sencillo y dinámico de aplicaciones web. Es una herramienta que implementa las técnicas AJAX .
- Prototype nos simplifica gran parte del trabajo cuando se pretende desarrollar páginas altamente interactivas.
- Funciones de Prototype
  - Función $(): Es un atajo a la función del DOM document.getElementById(),
  - Función $A(): Esta función convierte cualquier parámetro en un objeto array,
  - Función $H(): Convierte un objeto en un hash enumerable
  - Función $F(): Dado un ID, devuelve el valor de cualquier campo de formulario
  - Función document.getElementsByClassName(): Recibe una clase como parámetro y devuelve un vector con los elementos que tienen como atributo className de la clase.
  - Función $$(): recibe como parámetro un selector CSS y devuelve un vector con cada elemento que cumpla con el criterio del selector dado

6. Librerías de JavaScript: Prototype

- Trabajando el DOM
  - Prototype tiene algunos objetos (Element, Insertion, Observer y Position) que permiten distintas formas de manipular el DOM.
- Manejando formularios
  - Los objetos Form y Field prevén un número de funciones simples pero convenientes para trabajar con formularios y campos «input», así como el código que soporta la puesta en práctica de AJAX con Prototype.

6. Librerías de JavaScript: YUI

- Yahoo User Interface(YUI), una serie de bibliotecas escritas en JavaScript, para la construcción de aplicaciones interactivas (RIA). Liberadas bajo licencia BSD por parte de la compañía Yahoo. D
- ichas bibliotecas son utilizadas para el desarrollo web específicamente para ser usadas como la programación de aplicaciones de escritorio, con componentes vistosos y personalizables y con una amplia implementación con AJAX.
-

Ejemplo de JavaScript

- Javascript básico
  - http://www.w3schools.com/js/js_examples.asp
- Javascript con Objetos
  - http://www.w3schools.com/js/js_ex_objects.asp
- Javascript con las propiedades del Navegador
  - http://www.w3schools.com/js/js_ex_browser.asp
- Javascript DOM ejemplos
  - http://www.w3schools.com/js/js_ex_dom.asp

Bibliografía

- Librosweb.es
- Javascript: Conceptos básicos y avanzados. Informática Técnica. Ediciones ENI. Autor: Enmanuel Gutierrez.